

Python Class - Day 2

1. Revision of Previous Class (Keywords Recall)

You asked students to recall **keywords discussed in the previous class**.

Students mentioned the following concepts:

Web and Internet Basics

- API
- Domain name
- Frontend
- Backend
- Framework
- URL
- IP address
- Software
- Endpoint
- Method
- Network
- Router
- Database
- HTTP
- HTTPS
- Status code
- Request body
- Response body
- Validation
- Protocol
- Client
- Server

- Local vs Remote
- Internet
- WiFi
- Open source
- GitHub
- Applications

You told students:

These are **fundamental building blocks**. If you understand these, everything later will become easier.

2. Domain Name

Definition you explained

A **domain name** is:

- **Unique**
- **Human readable**
- Easier for humans to remember than IP addresses.

Example

google.com

Humans remember:

google.com

But computers communicate using:

IP address

3. IP Address

You explained that **every server on the internet has an IP address**.

Example given:

8.8.8.8

You asked students if they had seen it before.

Then explained:

| This IP address belongs to **Google's DNS server**.

4. How Domain Name Becomes IP Address (DNS Resolver)

You explained this using a **step-by-step story**.

Step 1

User types:

google.com

Step 2

Browser **does not understand domain names directly**.

It needs:

Public IP address

Step 3

Browser asks a **DNS Resolver**:

| “What is the IP address of google.com?”

Step 4

DNS resolver returns the IP address.

Example mapping:

google.com → 18.xx.xx.xx

Step 5

Browser sends request to that IP address.

5. DNS Resolver Concept

You explained that:

DNS resolvers are **servers whose job is to translate domain names into IP addresses**.

You said:

| There are **thousands of DNS resolvers in the world**.

Example famous DNS resolvers:

Google DNS

8.8.8.8

Cloudflare DNS

1.1.1.1

6. How DNS Is Assigned to Your Laptop

You explained the process when connecting to WiFi.

Step 1

When laptop connects to WiFi:

Router assigns DNS resolver

That DNS resolver may be:

- nearest server
 - ISP resolver
 - router configured resolver
-

7. How You Troubleshoot DNS Issues (Real Example)

You gave a **real troubleshooting story**.

Scenario:

Someone told you:

“Internet is not working on my laptop.”

Your diagnosis

You checked DNS settings.

By default:

WiFi router assigns DNS resolver

But that DNS resolver was **not working properly**.

Solution

You manually changed DNS.

Steps:

1. Go to network settings
2. Change DNS from

Automatic

to

Manual

3. Enter Google DNS
8.8.8.8

After that:

Internet started working again

8. How to Check DNS in Laptop

You explained how to check network configuration.

Command

ipconfig

Run this in:

Command Prompt

It shows:

- IP address
 - DNS resolver
 - network information
-

9. Endpoint

You asked:

| What is an endpoint?

Explanation given:

Endpoint means **location of a service**.

You simplified it as:

Endpoint $\approx$ URL

10. HTTP Methods

You asked students to name methods.

Examples mentioned:

GET

POST

These are used to **communicate with servers**.

11. Software Examples

You asked for software examples.

Student answered:

Firefox

Other examples could be:

- Chrome
- Edge

These are **applications used by users**.

12. Database Examples

You asked students to name databases.

Examples mentioned conceptually:

Database systems

Used to **store application data**.

13. HTTP vs HTTPS

You discussed protocols:

HTTP

HTTPS

These are **communication protocols between client and server**.

14. Status Codes

You asked students about HTTP status codes.

Example mentioned:

200

404

Meaning:

- **200 → Success**
 - **404 → Resource not found**
-

15. Request Body

You explained request body format.

Preferred format:

JSON

Example:

```
{  
  "name": "John",  
  "age": 25  
}
```

You also clarified:

It **does not have to be JSON**, but JSON is most commonly used.

16. Response Body

Server responses are usually sent in:

JSON format

Example:

```
{  
  "status": "success",  
  "message": "User created"  
}
```

17. Validation

You asked students:

| How many types of validation are there?

Answer:

17.1 Client Side Validation

Happens in:

browser

Example:

- required fields
 - input format checks
-

17.2 Server Side Validation

Happens in:

backend server

Example:

- checking database
 - business rules
-

Important Point You Emphasized

Developers should implement:

Both client-side and server-side validation

18. Client vs Server

You briefly discussed:

Client

Examples:

- Browser
- Mobile app

Server

Example:

- Backend application

Server:

- processes request
 - returns response
-

19. Local vs Remote

You mentioned:

Local

Running on your own machine.

Example:

localhost

Remote

Running on:

remote server

Example:

- cloud servers
 - production servers
-

20. GitHub

You asked students:

| Do you all have GitHub accounts?

Purpose of GitHub:

- store code
- share projects
- collaborate with team

You told students to **create GitHub accounts** if they don't have one.

21. Laptop Requirement

You asked students:

| Does everyone have a laptop?

You told them:

| From tomorrow everyone bring laptop.

Reason:

We will start installation and practical sessions.

22. Software Installation Preparation

You asked students if they have installed:

22.1 VS Code

Code editor used for development.

22.2 Python

You asked which version they have.

Students mentioned versions like:

3.14

You instructed them:

Install this version

Python 3.12

Specifically:

Python 3.12.10

You said you will **share the installation link**.

23. WhatsApp Group for Class

You discussed creating a group to share:

- installation links
- notes
- updates

You asked students to create a **separate group** and share the invite link.

24. Django Version Discussion

After discussing the WhatsApp group, you asked students:

| Do you know the latest version of Django?

Some students answered with versions like **Django 6**.

You then explained that:

Django is a **Python web framework** used to build:

- websites
- APIs
- backend systems

You clarified that before installing Django, we must ensure certain **requirements are ready**.

25. Requirements to Run Django

You explained that Django cannot run alone. Certain things must exist first.

The basic requirements you mentioned were:

1. **Python installed**
2. **Virtual Environment**
3. **Django package**

4. Code editor / IDE

Only after these are ready can we start creating Django projects.

26. Virtual Environment

You introduced the concept of **virtual environment**.

Students were first asked if they knew the term.

Then you explained it using a **real-life analogy**.

Virtual Environment Analogy (Human Survival Example)

You explained it like this:

For a **human to survive**, certain things are required.

Examples you mentioned:

- Heat
- Light
- Water
- Air

These together create a **suitable environment for survival**.

Cage Analogy

You then gave another example.

Imagine **a person inside a cage**.

Inside that cage we provide:

- specific temperature
- food
- water
- necessary conditions

That cage creates a **controlled environment**.

Mapping This to Programming

You explained:

A **virtual environment in programming is similar to that cage.**

Each project can have its **own isolated environment.**

Inside that environment we install only the packages needed for that project.

27. Why Virtual Environment Is Needed

You explained the problem when we **don't use virtual environments.**

Example scenario:

Project A requires:

Django 3

Project B requires:

Django 5

If both are installed globally, they may **conflict with each other.**

Virtual environments solve this problem by isolating dependencies.

28. Version Importance

You emphasized that **exact versions matter.**

Example:

Python 3.10

Django 4.2

Even small differences in version can cause:

- errors
- compatibility issues

So developers must carefully manage versions.

29. Package Installation

You explained how developers install packages in Python.

The tool used is:

pip

Example command:

pip install django

But you emphasized that **installation should be done inside the virtual environment**.

30. Activating Virtual Environment

Before installing packages, the environment must be **activated**.

Example command (Windows):

Scripts/activate

Typical workflow you described:

cd project_folder

Scripts/activate

pip install django

Once activated, packages install **inside the environment only**.

31. Requirements File

You explained that projects usually maintain a file listing all dependencies.

This file is called:

requirements.txt

Example content:

Django==4.2

djangorestframework==3.14

This ensures everyone uses the **same versions**.

Installing Using Requirements File

Command:

pip install -r requirements.txt

This installs **all packages listed in the file.**

32. Integrated Development Environment (IDE)

You asked students:

| What is IDE?

Then explained:

IDE stands for:

Integrated Development Environment

It is a **tool that helps developers write code.**

Examples mentioned:

- Notepad
 - VS Code
 - PyCharm
-

Important Clarification You Gave

IDE is mainly used to:

- write code
- manage files
- assist development

Actual execution of code is handled by:

Python interpreter

33. How Programming Languages Run

You explained that computers understand only:

Machine language

But humans write programs using:

High level languages

Therefore a **translator is required.**

34. Types of Translators

You mentioned two types:

Compiler

Examples:

- C
- C++

These convert the entire program before execution.

Interpreter

Example:

Python

Python executes code **line by line**.

35. Python Basics Introduction

You then began discussing **basic Python concepts**.

You explained that Python is a:

High level programming language

And also:

Dynamically typed language

36. Dynamic Typing

You explained that Python automatically detects variable types.

Example:

```
x = 10
```

Python automatically understands that **x is an integer**.

No explicit type declaration is needed.

37. Python Data Types

You listed the fundamental data types.

Integer

Whole numbers.

Examples:

10

25

100

Float

Decimal numbers.

Examples:

10.5

3.14

String

Text data.

Examples:

"Hello"

"Abdul"

Boolean

Two possible values:

True

False

Used in logical conditions.

38. Character Example

You also showed examples like:

'A'

Single characters.

39. Python Collections

You then moved to collections.

Collections store **multiple values together**.

List

Example:

[1,2,3,4]

Characteristics:

- ordered
 - mutable
 - duplicates allowed
-

Tuple

Example:

(1,2,3)

Difference from list:

- immutable
 - values cannot be changed
-

Set

Example:

{1,2,3}

Important rule you explained:

Sets **do not allow duplicate values**.

Example:

```
{1,1,2,3}
```

Result becomes:

```
{1,2,3}
```

40. Loops

You introduced loops by explaining the idea of **repeating tasks**.

Example without loop:

```
print("Hello")
```

```
print("Hello")
```

```
print("Hello")
```

This is repetitive.

Using Loop

Example:

```
for i in range(3):
```

```
    print("Hello")
```

Output:

```
Hello
```

```
Hello
```

```
Hello
```

Loops reduce repetitive code.

41. End of Class Recap

At the end, you asked students to summarize what was discussed.

Topics covered included:

- DNS

- Domain name
 - IP address
 - GitHub
 - Virtual environment
 - Python basics
 - IDE
 - Data types
 - Collections
 - Loops
 - Package installation
-

Complete Topic Flow of the Class

1. Revision of previous keywords
2. Domain name
3. IP address
4. DNS resolution
5. DNS troubleshooting
6. `ipconfig` command
7. Endpoint
8. HTTP methods
9. Protocols
10. Status codes
11. Request body
12. Response body
13. Validation
14. Client vs server
15. Local vs remote
16. GitHub
17. Laptop requirement
18. Python installation

19. [WhatsApp group](#)
20. [Django introduction](#)
21. [Virtual environment](#)
22. [Dependency isolation](#)
23. [Package installation](#)
24. [Requirements.txt](#)
25. [IDE concept](#)
26. [Compiler vs interpreter](#)
27. [Python basics](#)
28. [Data types](#)
29. [Collections](#)
30. [Loops](#)